Surgical Inference: Decomposing Generative Models for Heterogeneous On-Device SoC Inference

Matt Mireles
Independent Researcher
mattmireles@gmail.com

October 23, 2026

Abstract

On-device generative inference usually means choosing one backend, and single-backend deployment leaves most of a heterogeneous SoC idle. We present *Surgical Inference*: compile a generative model as a set of fixed-shape submodels and native stages, reshape each stage — without changing its math — until the engine that should run it admits it, and place every stage on the CPU, GPU or Neural Engine (ANE) by measurement.

On Kokoro-82M text-to-speech, the ANE rejects the vocoder, 89% of the model's FLOPs, on tensor geometry. Folding its time axis into overlapping rows, rewriting each rejected op into an admitted equivalent and keeping exact global normalization statistics puts all but one of its 1,928 ops on the ANE. Splitting each vocoder stage so the ANE runs the convolutions while the GPU runs normalization and activations, over shared memory, keeps both engines busy at once. The pipeline synthesizes 27 s of speech in 309 ms on an M1 Mac Mini, 9.8× faster than MLX on the same machine, and in 319 ms on an iPhone 15 Pro Max, at 40.8–45.8 dB SNR against fp32. It needs 47–53% less peak memory than MLX on the Mac at 30 s and 58–95% less on the iPhone clips MLX completed, and on an iPhone 15 Pro Max it stays thermally nominal where MLX, on the same 30 s work, reaches throttling. On Magenta RealTime 2, a streaming music model with a 40 ms frame deadline, the same procedure runs faster than real time on an iPhone 15 Pro Max in bounded runs, though not yet in a 10-minute soak, and finds two more structural admission rules: in-graph state mutation, not attention, is the ANE compile cliff, and tensor layout decides whether FP16 output stays finite on each engine.

Negative results are reported as results: no valid monolithic export, a list of optimizations that failed, and an estimated ceiling for the exact model below 90% of the hardware floor. Code, models and benchmarks are open source.

Keywords: on-device inference, heterogeneous computing, Apple Silicon, Neural Engine, Core ML, model decomposition, text-to-speech, vocoder, autoregressive streaming

1 Introduction

Apple Silicon puts CPU cores, a GPU and a neural accelerator (the Apple Neural Engine, ANE) on one die with unified memory, and similar CPU/GPU/NPU designs now ship in most phones and laptops. Local generative inference nonetheless runs almost entirely on the GPU, through llama.cpp, MLX and PyTorch MPS. The GPU is programmable and forgiving, but a GPU-only model competes with rendering, media and any concurrent local LLM, while the ANE sits idle.

The obvious alternative — compile the whole model for the ANE — fails for most generative models. ANE compilers impose static shapes, an op set and per-axis tensor limits, and a generative

model is rarely one homogeneous graph: it mixes dense convolutions, recurrence, data-dependent indexing, DSP and sampling. When the compiler rejects part of the graph, Core ML falls back silently to another engine, and the developer learns nothing.

We argue that the unit of deployment should be the inference pipeline, not the model file, and that a rejection by the accelerator is a statement about a tensor's *shape*, not about the model. *Surgical Inference* cuts a pipeline at its dataflow seams into fixed-shape Core ML submodels and native stages, reshapes stages the ANE rejects into equivalent forms it admits, and places each stage by measurement. We test it on two workloads chosen to be structural opposites: Kokoro-82M, a batch text-to-speech pipeline, and Magenta RealTime 2 (MRT2), an autoregressive music model that streams under a 40 ms per-frame deadline on a phone.

The paper's thesis emerged from the two case studies rather than preceding them. **In both workloads, what decided whether the ANE admitted a stage — and whether its output stayed correct — was structure the developer controls: tensor geometry, state boundaries and memory layout, not which ops the model uses.** Kokoro's vocoder was rejected on geometry and admitted after folding; MRT2's transformer was rejected for mutating state in-graph and admitted as a stateless step; MRT2's codec produced NaNs in one layout and correct audio in another. Op-level rewrites suggested by ANE folklore changed nothing in either.

We make five contributions:

1. **A decomposition methodology** for multi-stage generative pipelines: a three-motif classification (Sequential-Dynamic, Data-Dependent Logic, Dense-Static), a decomposition procedure with a per-stage placement ablation, and host-glue validation rules (§3). It systematizes practice already visible in per-stage Core ML deployments such as Apple's Stable Diffusion port and WhisperKit (§7); we make the procedure explicit and test it on two opposite workloads.

2. **Folding an audio-rate vocoder onto the ANE without changing its math** (§4.3). Kokoro's vocoder carries time axes of up to 720,000 elements against a 16,384-element per-axis limit. Folding time into rows with overlapping halos, five op rewrites and exact global AdaIN statistics make it ANE-resident and exact (68 dB against the unfolded generator in fp32).

3. **One stage, two engines** (§4.4). Measured per op class, the ANE is bound by the vocoder's elementwise work and the GPU by its convolutions. A hybrid generator runs every convolution on the ANE and normalization and activations in Metal, over shared IOSurface tensors, with both engines busy at once. It beats the all-ANE form by 16–78% at every input of 7 s and longer on all five devices, at a measured cost in phone memory and GPU time (§4.5.4).

4. **Structural ANE admission findings** from the streaming case study (§5): in-graph state mutation, not attention math, is the compile cliff for streaming transformers; internal layout decides FP16 numerical survival per compute unit; and weight-bound stages cost weight bytes ÷ DRAM bandwidth per call on every engine.

5. **Measured results, a cumulative ablation and negative results** on three Macs and two iPhones under a frozen protocol (§4.5): a one-change-at-a-time ladder from the previous pipeline to the current one, peak memory and phone thermal state against MLX, a lab estimate of the remaining distance to the hardware floor, a monolithic-export control that fails, and the optimizations that did not work.

Code, pre-converted Core ML models and benchmark data for Kokoro are at `https://github.com/mattmireles/kokoro-coreml` and `https://huggingface.co/mattmireles/kokoro-coreml`. MRT2's conversion and validation layer is at `https://github.com/mattmireles/`

magenta-realtime-2-iphone; its streaming runtime lives in a fork that is private at the time of writing (Appendix A).

2 Background

The SoC. Apple's M- and A-series chips share one DRAM pool among three engines with different admission rules and power profiles. The *CPU* handles branching, dynamic shapes and DSP (Accelerate/vDSP). The *GPU* runs arbitrary compute through Metal and handles dynamic shapes. The *ANE* is a fixed-function accelerator for convolutions, matrix multiplies, elementwise ops, pooling and normalization, under strict constraints: static shapes, a restricted op set, no data-dependent control flow, and per-axis tensor limits (16,384 elements per axis, by community documentation [20, 32] and our own compiler errors). Core ML [3] dispatches to all three, but a compute-unit setting is a *request*: subgraphs the ANE rejects run elsewhere without an error.

GPU-centric practice. llama.cpp [11], MLX [13] and PyTorch MPS [5, 31] target only the CPU and GPU. The common view that Core ML offers little for generative inference is often right for large language models, which are bandwidth-bound; the ANE adds no bandwidth. It does not transfer to pipelines with separable dense, sequential and data-dependent regions.

Kokoro-82M [14, 15] is an Apache-2.0 text-to-speech model built on StyleTTS 2 [22] with an iSTFTNet-style vocoder [18]. Its reference inference path has seven stages: duration prediction (a BERT encoder and an LSTM prosody predictor), duration-dependent expansion of tokens to frames, pitch and noise (F0/N) prediction, decoder pre-processing (*DecoderPre*), a harmonic source (hn-nsf: SineGen plus STFT), the vocoder (*generator*), and trim. The generator holds 1,348 of the model's 1,520 GFLOP for a 27 s utterance (89%).

Magenta RealTime 2 [23, 25] is Google's open-weights live-music model. A decoder-only transformer generates one frame of 12 residual-vector-quantized (RVQ) tokens every 40 ms, and a SpectroStream codec decoder in the SoundStream lineage [36] turns each frame into 48 kHz stereo audio. Upstream real-time inference runs through MLX on M-series Macs only. We use `mrt2_small` (230M parameters).

3 Surgical Inference

Surgical Inference changes the compiled artifact boundary, not the weights. A reference model becomes a few fixed-shape neural packages plus native code for dynamic or irregular work. The rules are short: cut only at real dataflow boundaries; preserve numerical equivalence at each cut; when the accelerator rejects a stage, change the stage's shape before giving up on the accelerator; let measured placement override intuition; and treat host glue as part of the model.

3.1 Computational Motifs

Sequential-Dynamic (SD): recurrence, autoregressive decoding, sampling loops. Candidate engines: CPU or GPU. The motif belongs to the boundary drawn, not to the stage inherited: when the only recurrence is cache state, the per-step math is Dense-Static and the bookkeeping is host logic (§5.1).

Data-Dependent Logic (DDL): computation whose structure depends on tensor values — duration expansion, masking, top-k, trimming. Candidate engine: native CPU code; too irregular for an accelerator to amortize.

Dense-Static (DS): convolutions, matrix multiplies, normalization and elementwise ops with compile-time shapes. Candidate engines: Core ML, with placement measured. Dense-Static is the candidate set for the ANE, not a guarantee of admission (§6.1).

3.2 Procedure

1. Trace the reference forward pass; mark boundaries where data-dependent logic enters or leaves the graph, values stable enough to validate, and independent branches.
2. Classify each stage by motif.
3. **Dense-Static stages:** make shapes static (bucketing with masks), export to Core ML, and validate against a native-length fp32 reference across bucket fills, by SNR over the valid region (per-stage correlation on padded inputs proved too weak, §4.1). If the ANE rejects the stage, find the binding structural property (§6.1) and reshape: fold long axes into rows, move state to host-owned inputs and outputs, change internal layout. Then ablate placement per stage — `.cpuAndNeuralEngine`, `.cpuAndGPU`, and splits within the stage by op class — and keep the fastest variant that passes validation. Prove placement with `MLComputePlan` or Instruments; never trust `.all`.
4. **Data-Dependent stages:** implement natively (Swift/C, Accelerate). Profile them: a naive port is often slower than PyTorch.
5. **Sequential-Dynamic stages:** if the recurrence is only cache state, export a stateless step function with caches as inputs and updates as outputs. If the stage is weight-bound, expect per-call cost $\approx$ weight bytes $\div$ DRAM bandwidth on any engine and reduce calls before chasing placement (§5.3).
6. Orchestrate natively. Overlap independent stages across engines where the dataflow proves it safe. Validate every host-side consumer of a model output against the reference at least once, read tensors through their strides, and prove that stateful graphs read their state (§5.4).
7. Benchmark end to end against the original framework, GPU-native frameworks, other ports, and a monolithic conversion if one can be built (§4.5.8), across input lengths and devices; change one thing at a time when attributing gains.

Host glue is part of the model. Several of the largest costs we found were outside any model graph: a one-hot alignment implemented as a dense matrix multiply, boxed element reads over strided FP16 outputs (about 1 s per 30 s clip on an M1 until replaced by vectorized run conversion), and Core ML repacking pointer-backed inputs on every call, which doubled the per-call cost of ANE convolutions until the tensors were backed by IOSurfaces (§4.4). In MRT2, host glue silently destroyed correctness (§5.4). End-to-end wall time is the primary metric; per-stage times are diagnostics.

4 Case Study 1: Kokoro-82M

4.1 Decomposition

Figure 1 shows the decomposed pipeline: four Core ML model families — duration, F0Ntrain, DecoderPre and the generator — and three Swift stages, with no Python at inference time. Table 1 gives the shipped placement.

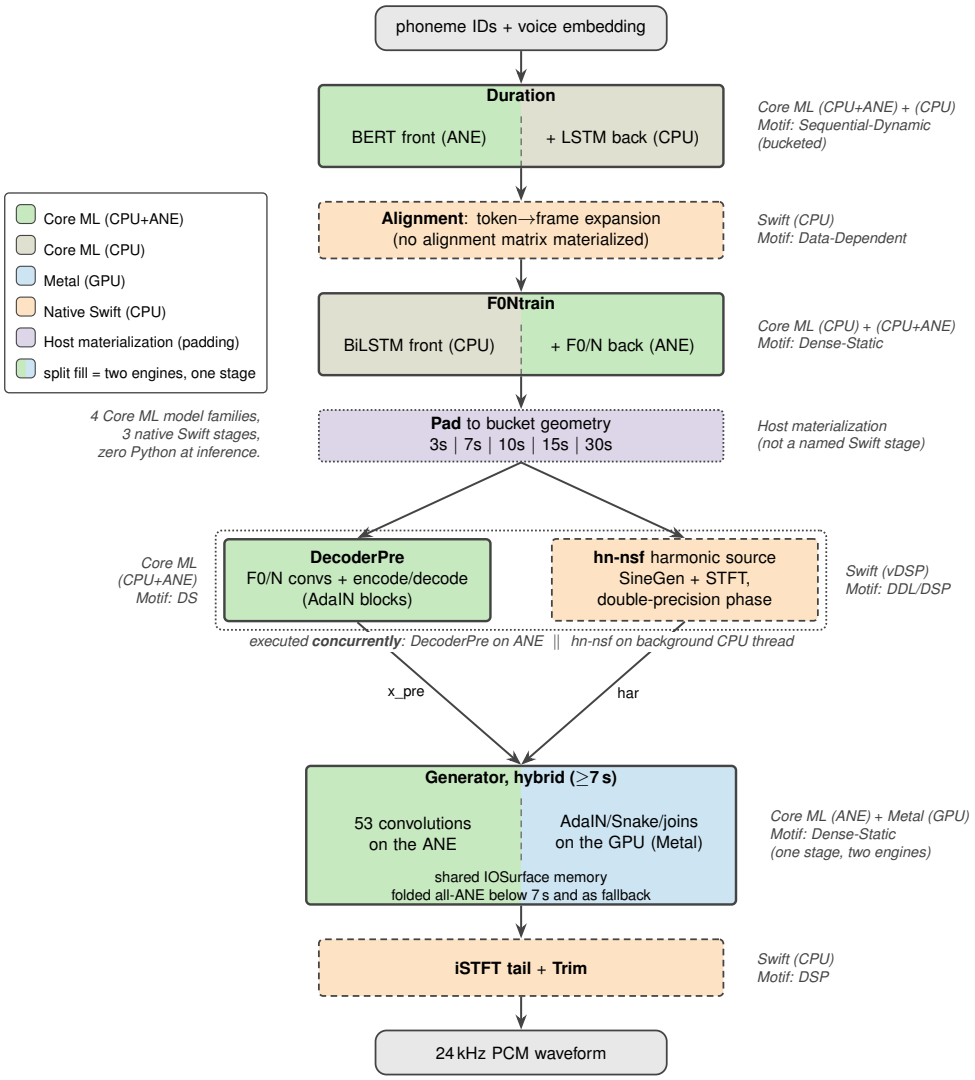


Figure 1: The decomposed Kokoro-82M pipeline: four Core ML model families and three native Swift stages. Color is placement. Duration and F0Ntrain are split between the CPU and the ANE; the generator runs its convolutions on the ANE and its normalization and activations on the GPU (hybrid, 7 s and longer) or entirely on the ANE (folded, below 7 s). DecoderPre and the hn-nsf harmonic source run concurrently on different engines.

Export. Packages are exported with coremltools 8.3 [3, 4] in FP16. Duration and F0Ntrain ship at enumerated sizes with masks: the ANE rejects flexible shapes, an LSTM over a symbolic length ran about 5× slower on the CPU, and `EnumeratedShapes` cost about 140 ms whenever the shape changed. Replacing an unrolled bidirectional LSTM (up to 134,397 ops) with two stock `lstm` ops cut the padded duration stage from 82–733 ms to 11–45 ms on an M2 Ultra. Masking matters for correctness, not just speed: bucket padding had contaminated normalization statistics (one 2.8 s utterance forced through every bucket sat at 15 dB F0 SNR against a native-length run), which per-stage correlation above 0.9999 had not revealed. Current packages are scored by SNR over the valid region against fp32 PyTorch on real inputs.

Native stages. Alignment copies each token's vector into its predicted frame slots; no alignment matrix exists in the hot path. The harmonic source runs in Swift with vDSP: double-precision

Table 1: Shipped placement per stage. October 2026 is the previous pipeline (the ablation's R0, §4.5.3).

Stage	Motif	Now	October 2026
Duration (BERT + LSTM)	SD	BERT on ANE, LSTMs on CPU (ANE-layout split)	CPU+GPU
Alignment	DDL	Swift: direct token-to-frame expansion	same
F0Ntrain	DS	split CPU + ANE	CPU+GPU
DecoderPre	DS	ANE, ANE-friendly layout	ANE
hn-nsf harmonic source	DDL/DSP	Swift/vDSP, concurrent with DecoderPre; noise made during F0Ntrain	concurrent with DecoderPre
Generator	DS	hybrid: convolutions on ANE, AdaIN, Snake and joins on GPU ($\geq$7 s); folded, all ANE, below 7 s	GPU (fixed buckets; flexible on iPhone)
iSTFT tail and trim	DSP	Swift, parallel	in the generator graph

phase accumulation, phase increments computed per F0 frame instead of upsampling by 300 and back, bulk noise generation, and a cached windowed-DFT STFT. An early port was 4× slower than PyTorch until per-sample RNG calls through Swift's protocol dispatch were replaced (a 12× speedup); that and an absorbing zero state in its xorshift RNG are the kind of defect the procedure's profiling and validation steps exist to catch. hn-nsf cannot itself be exported: Core ML's execution of SineGen's cumulative phase gives correlation near zero.

4.2 The Vocoder Wall

The generator's harmonic input has a last axis of 14,401–144,001 elements across buckets, and its waveform output 72,000–720,000. Loaded for the ANE, it fails compilation on every Mac we tested (`ANECCompile() FAILED`; `Tensor width goes beyond limit supported (16390 > 16384)`); under `.all`, macOS reroutes it silently, and in an April 2026 ablation the fallback ran 54× slower than CPU+GPU. Source rewrites suggested by ANE reverse-engineering literature [20, 32] — `Linear` to 1×1 convolution, native instance-norm lowering, broadcast AdaIN, FP16 inputs, palettization — all validated and none changed runtime. The October 2026 pipeline therefore ran the generator on the GPU, where at 30 s it took 88.5% of end-to-end time on the M2 Air and 92.7% on the A14 (R0 in Table 9). laishere/kokoro-coreml [21] splits the vocoder into several Core ML packages; under `.cpuAndNeuralEngine` on an M1, `MLComputePlan` assigns about 47.5% of its estimated vocoder cost to the ANE and the rest to the CPU.

4.3 Folding the Vocoder onto the ANE

The limit is on one axis's length, not on the work. The generator is a stack of 1-D convolutions with a receptive field of about 8 `x_pre` frames, so its time axis can be cut into rows that the ANE processes in a batch axis. Each row carries `core` frames plus a `halo` of 8 frames on each side; outputs are taken from the core and stitched back. Table 2 lists what else had to change; every rewrite is exact in fp32 (68 dB against the unfolded generator).

Table 2: Making the folded generator admissible. Op admission found by bisection with `MLComputePlan`, one op per model, on an M2-generation ANE.

Obstacle	Exact rewrite
Time axis > 16,384	Rows of `core` frames plus 8-frame halos in the batch axis; a 15,605-wide single row still failed, 8,165 compiles
`ConvTranspose` upsamplers rejected	Polyphase convolution to $C_{\text{out}} \cdot s$ channels plus a width shuffle
Stride-6 noise convolution rejected	Space-to-depth, then a plain convolution
Dilation-5 convolution rejected	Reshape width to $(W/5, 5)$ with rows in the batch axis and a $(11, 1)$ kernel; later, phase-major storage turns every dilated convolution into a plain one
`sin` rejected on M1/M2/A14 ANEs	Snake folded to $z + \sin^2(z)$ (the scale moves into the next convolution's weights); a degree-3 polynomial on chips without ANE `sin`
AdaIN over rows	Statistics mask (each global position counted once, by its owner row) and zero mask; exact *global* statistics per channel
iSTFT tail	Graph stops at `conv_post` logits; exp/sin/iSTFT run in Swift

Two numerical problems surfaced only on the ANE. First, some channels have mean ≈3 and standard deviation ≈0.08; the ANE's FP16 masked mean was about 10^{-3} relative off (the GPU's 2×10^{-4}), shifting whole channels by 0.04σ and holding the first build to 38.7 dB. Two-pass centring ($c = x - \bar{x}$; $c = c - \bar{c}$) fixed it. A pre-scaling meant to avoid FP16 overflow instead pushed small variances into subnormals, which the ANE flushes to zero, and the output fell to 16 dB — visible only when blocks were composed. Second, AdaIN statistics must be exact and global: estimating them from every k-th row, which would have allowed streaming rows with bounded memory, gave 1–4 dB for every k from 2 to 10.

The frozen folded package (polynomial Snake, 30 s) places 1,927 of its 1,928 ops on the ANE of both an M2 Air and an A14, and scores 40.8–44.1 dB against the fp32 PyTorch generator on the frozen test clips; the native-`sin` variant, which the runtime picks only on A17-class ANEs, leaves 96 `sin` and multiply ops on the CPU of an M2. Admission did not make it fast everywhere. In the rerun's ablation the folded generator beat the GPU generator on the A14 (30 s pipeline: 1,556 against 3,679 ms) and on the M2 Air (821 against 1,091 ms), but on the M2 Ultra, whose GPU is large, the folded pipeline took 706 ms at 30 s, against 373 ms for the October GPU pipeline under similar load in another session (287 ms in its quiet pass). Admission was necessary, not sufficient.

4.4 One Stage, Two Engines

Ablating the folded graph op class by op class explains why. In lab runs on earlier builds, on an M1 ANE Snake activations took about 43% of the generator's time, AdaIN 23% and the convolutions 23%; on the A17 Pro, the convolutions alone ran in 514 ms on the ANE and 1,677 ms on the GPU. The ANE is bound by the vocoder's elementwise work; the GPU by its convolutions. The engines have complementary weaknesses inside one stage.

The hybrid generator assigns each op class to the engine that is fast at it. The ANE runs every convolution, as 53 small Core ML functions per row count; Metal kernels run AdaIN statistics, the affine, Snake and the branch joins. Three engineering points made it work:

- **Shared memory without repacking.** With pointer-backed `MLMultiArray` inputs, Core ML repacked tensors on every call and an ANE convolution took 10.2 ms on an M1; backed by IOSurface pixel buffers, which Metal reads in place, it took 5.7 ms. The "wake-up" penalty we had attributed to ANE power gating was this copy.
- **Layouts both engines like.** Dilated convolutions read and write rows phase-major (sample $q \cdot d + p$ stored at $p \cdot W/d + q$), so on the ANE they become plain convolutions, and the Metal passes convert layout inside work they already do. Vectorizing those passes to `half4` brought them to about the memory-bandwidth floor.
- **Overlap.** Independent chains keep the ANE and GPU busy at once: on the A14, a 30 s generator call on the frozen build spans 375 ms with the ANE busy for 292 ms and the GPU for 268 ms, both at once for 192 ms.

The hybrid scores 40.8–45.8 dB; its worst clip, a 1.2 s "Hi.", runs on the folded generator. It serves buckets of 7 s and longer; at 3 s its roughly 100 ANE/GPU round trips lose to the single folded call. If a Metal command buffer fails — as when an iOS app is backgrounded and loses GPU access — the executor re-runs that clip on the folded generator.

4.5 Evaluation

Hardware. M1 Mac Mini (16 GB, 68 GB/s), M2 MacBook Air (24 GB, 100 GB/s), M2 Ultra Mac Studio (64 GB, 800 GB/s), iPhone 12 Pro (A14, 6 GB) and iPhone 15 Pro Max (A17 Pro, 8 GB). The M2 Ultra ran macOS 26.7.1, the M2 Air and M1 macOS 27.0.1, and both iPhones iOS 27.0.1.

Inputs and boundary. Five frozen inputs targeting the runtime buckets (2.80, 6.75, 9.60, 13.90 and 27.38 s of audio), voice `af_heart`, speed 1.0. Timing runs from token IDs and voice embedding in to a 24 kHz waveform in memory (Appendix B).

Protocol (plan 014, fixed before the run, Appendix A; deviations below). Release builds. Each Surgical cell discards three preflight calls, records one cold call, and reports the median of ten warm calls; the Mac MLX harness records one cold call and ten warm calls with no preflight. Macs: plugged in, fleet worker paused, and each arm waits for the one-minute load average to fall to 1.5. No Mac reached that gate (one-minute load 2–7 from background apps, with no fleet jobs running), so every Mac row is an upper bound. iPhones: one app launch per arm per bucket, a cooldown before each launch, a nominal thermal state at launch, thermal state recorded per call, and arm order alternated by bucket. Six cells ended above nominal and are marked where they appear: the A14's R0, R1 and R2 cells at 30 s (old generator) and the A17's MLX cells at 10, 15 and 30 s. *Deviations,* made mid-run to finish in one day: the phone cooldown fell from ten minutes to three (six for the A17's last six cells) and the Mac gate wait from 20 minutes to two. On the A14, the folded 3 s cell at three minutes took 98.5 ms against 92.0 and 98.2 ms for two same-pipeline cells at ten minutes, so the A14's launch-to-launch noise is at least 6.7%; the A17 had no clean ten-minute reference. The A17's first cells, taken while the phone was in use, were discarded and rerun, and the iPhone app was rebuilt once to restore MLX's model files, with Core ML code and packages unchanged. Phones are reported in this sustained regime; a single request after idle is slower (§4.6). MLX runs in the same session as the Surgical pipeline. Raw per-call timings, package hashes, commit and host state for every cell are archived (Appendix A).

Configurations. *Surgical* is the shipped pipeline (Table 1) with its default generator, hybrid (folded below 7 s), chosen by a rule fixed before the run: hybrid becomes the default only if it is faster than folded on every device, at equal quality, with no crash in the matrix. *MLX* is mlx-audio [9] 0.4.3 (commit `862dfbe`) on Macs and mlalma/kokoro-ios [26] 1.0.8 on iPhones. Only public Core ML APIs are used, except in the one column labeled *e5rt* (§4.5.7).

4.5.1 Headline results

Table 3: Warm median end-to-end wall time (ms), Surgical (hybrid default; folded below 7 s) vs MLX, Macs. × is MLX time over Surgical time. Every Mac missed the load gate, so all rows are upper bounds. "err": the pinned mlx-audio fails every 3 s input with a broadcast-shape error. ‡The M2 Air was the noisiest host (load 3–5) and its MLX ran about 2.4× slower than in October 2026, so its ratios are not comparable and are not used in any claim.

	M2 Ultra			M2 Air			M1 Mini		
Input	Surg.	MLX	×	Surg.	MLX	×	Surg.	MLX	×
3s	73.6	err	–	73.0	err	–	94.9	err	–
7s	53.7	189.1	3.5	71.9	1431.6	19.9‡	98.9	800.5	8.1
10s	70.6	272.5	3.9	240.1	1817.5	7.6‡	123.8	1102.9	8.9
15s	88.9	340.4	3.8	272.9	2426.3	8.9‡	168.5	1542.3	9.2
30s	158.9	769.8	4.8	377.4	5186.7	13.7‡	308.5	3014.6	9.8

Table 4: iPhone Release-build warm medians (ms), sustained regime. e5rt is the private-API path (§4.5.7); it is not used in any ratio. "OOM": the memory watchdog killed kokoro-ios on the 6 GB A14, twice. §The MLX cell ended above a nominal thermal state ("fair" at 10 and 15 s, "serious" at 30 s), so its time may include thermal slowdown and the ratio is an upper bound.

	iPhone 15 Pro Max (A17 Pro)				iPhone 12 Pro (A14)			
Input	Surg.	MLX	×	e5rt	Surg.	MLX	×	e5rt
3s	61.8	551.0	8.9	58.7	98.2	911.1	9.3	97.7
7s	103.2	1226.8	11.9	125.6	144.8	2050.3	14.2	142.1
10s	155.9	1717.1§	11.0§	188.6	185.7	2814.4	15.2	187.8
15s	223.8	2319.3§	10.4§	245.3	260.9	3950.8	15.1	264.3
30s	318.5	4797.3§	15.1§	335.0	479.9	OOM	–	454.7

The 27.38 s input synthesizes in 159 ms on the M2 Ultra, 309 ms on the M1 Mini, 319 ms on the iPhone 15 Pro Max and 480 ms on the iPhone 12 Pro: 172, 89, 86 and 57× real time. Against MLX on the same device and session, the pipeline is 3.5–4.8× faster on the M2 Ultra, 8.1–9.8× on the M1, 8.9–15.1× on the A17 Pro and 9.3–15.2× on the A14 (Figure 2). The pinned mlx-audio fails every 3 s Mac input, and kokoro-ios is killed by the memory watchdog at 30 s on the 6 GB A14; both failures reproduce the October 2026 run. Against the October 2026 run (another session, quieter Macs, so indicative only), one cell regressed: on the M2 Ultra the 3 s input, which runs the folded generator, takes 74 ms against 64 ms for October's GPU generator under similar load (45 ms in its quiet pass; Appendix C); on every other device the 3 s input got faster. The A17 Pro's ratios at 10 s and longer are upper bounds, because its MLX cells ran hot.

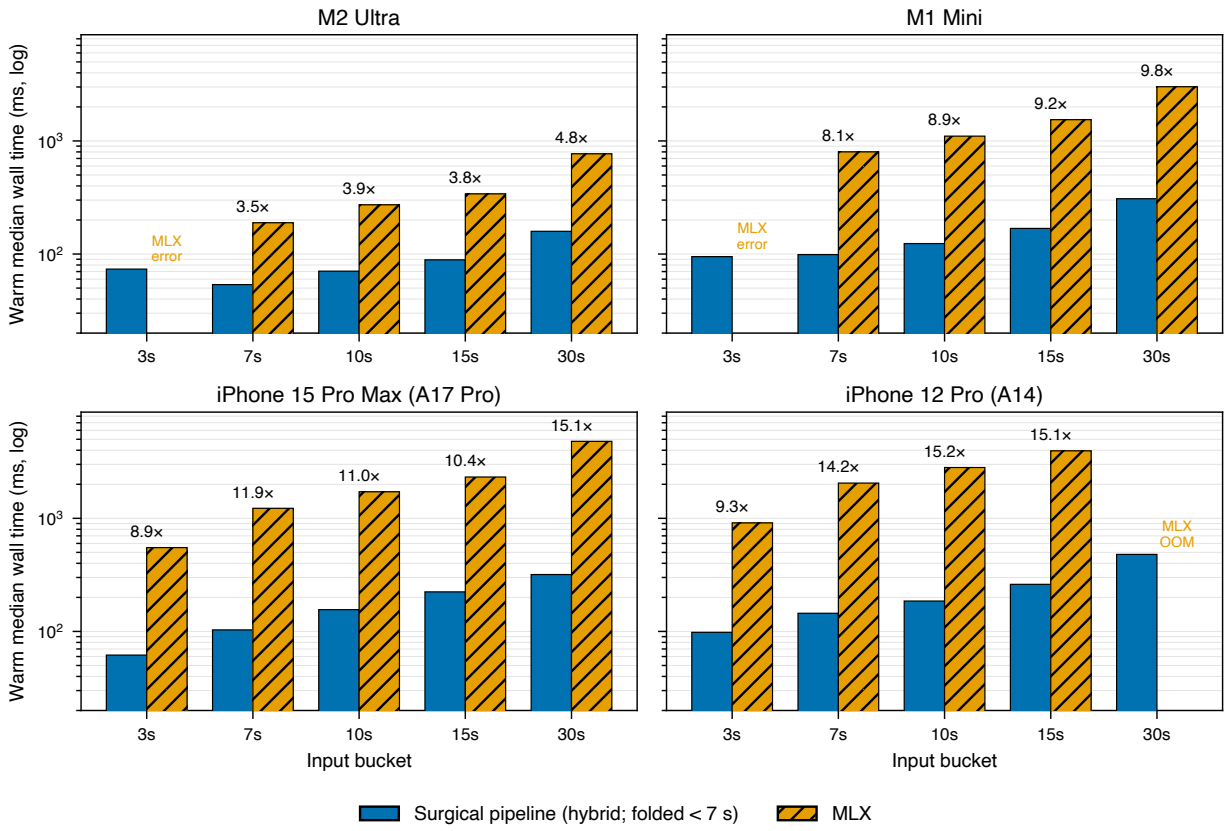


Figure 2: End-to-end warm median latency, the shipped pipeline vs MLX on the same device and session (log scale), from Tables 3 and 4. The M2 Air is omitted because its host was too loaded for a fair ratio. The pinned mlx-audio fails every 3 s Mac input, and kokoro-ios is killed by the memory watchdog at 30 s on the A14.

4.5.2 Memory and heat

The same run recorded peak memory for every cell and the iPhone's thermal state at every call. Neither was an endpoint of the plan fixed before the run, so we report both as observations.

On the M2 Ultra and the M1, MLX's peak is nearly flat across the inputs it completes (771–815 MB): its weights and runtime dominate. (The M2 Air, left out as the noisiest host, gives the same cuts: 77% less at 7 s, 53% less at 30 s.) The pipeline's grows with the input, from 134–144 MB at 3 s to 375–434 MB at 30 s, so it uses 70–77% less memory than MLX at 7 s and 47–53% less at 30 s. Percent less is the share of MLX's peak the pipeline does not use, rounded to the nearest percent. On the iPhones the gap is wider: MLX peaks at 1,182–3,192 MB, so the pipeline uses 58–79% less memory on the A17 Pro and 80–95% less on the A14 where MLX completes; at 30 s MLX does not fit on the 6 GB A14 at all, while the pipeline peaks at 831 MB. On the phones the hybrid generator is what costs memory (§4.5.4): at 30 s the folded generator peaks at 780 MB on the A17 Pro and 284 MB on the A14, against 1,338 and 831 MB for the hybrid.

The pipeline ended every cell at `nominal` on both phones, with either generator. MLX ended at `serious`, the state in which iOS throttles, at 30 s on the A17 Pro, and did so again in the separate memory session, launched about 70 minutes after the phone's previous run, with an unchanged median (4,806 against 4,797 ms). Its `fair` endings at 10 and 15 s did not repeat there, though

Table 5: Peak memory (MB) for one synthesis, shipped pipeline vs MLX, same inputs and session as Tables 3 and 4. Macs: maximum resident set size of a fresh process per cell; the M2 Air is omitted, as in Table 3. iPhones: peak `phys_footprint` over one app launch per cell. MLX fails every 3 s Mac input (its 3 s process peak is shown in parentheses), and is killed at 30 s on the A14, a 6 GB phone. The iPhone MLX cells were measured in a separate session with the same metric; Surgical cells re-measured there matched within 1.4%.

	M2 Ultra		M1 Mini		A17 Pro		A14	
Input	Surg.	MLX	Surg.	MLX	Surg.	MLX	Surg.	MLX
3s	144	(778)	134	(763)	255	1219	61	1182
7s	235	786	177	771	551	1791	282	1758
10s	263	796	199	777	655	2045	349	2006
15s	275	791	247	782	869	2481	500	2444
30s	434	815	375	801	1338	3192	831	OOM

Table 6: iOS thermal state (`ProcessInfo.thermalState`) at the end of each cell: 14 back-to-back syntheses of the same input, every cell starting at `nominal`. "Folded" is the all-ANE generator; at 3 s the shipped pipeline runs it too. In a second session, MLX again ended 30 s on the A17 Pro at `serious`, but ended 10 and 15 s at `nominal`.

	iPhone 15 Pro Max (A17 Pro)			iPhone 12 Pro (A14)		
Input	Surgical	Folded	MLX	Surgical	Folded	MLX
3s	nominal	nominal	nominal	nominal	nominal	nominal
7s	nominal	nominal	nominal	nominal	nominal	nominal
10s	nominal	nominal	fair	nominal	nominal	nominal
15s	nominal	nominal	fair	nominal	nominal	nominal
30s	nominal	nominal	**serious**	nominal	nominal	killed (OOM)

one 10 s attempt there, which started at `fair` against the start rule, ended at `serious` and was rerun. Cooldowns were not symmetric in the main run (some MLX cells got three minutes where pipeline cells got six), and the phone sometimes read `fair` a few minutes after a pipeline cell that had ended `nominal`, which may be heat left from earlier MLX cells. On the A14 MLX stayed `nominal` through 15 s and was killed at 30 s. The previous pipeline also heated the A14: its 30 s cell (R0, Table 7) ended at `serious`. Two cautions bound this finding. First, it compares the heat of the same work, not of the same time: MLX needs 9–15× longer for the same 14 syntheses, so the phone runs it longer. Second, the thermal state is a coarse four-level signal, recorded at every call and reported at the end of each cell, and we have no power measurements for Kokoro (§6.4); we therefore do not claim an energy saving. A paired power capture of the two pipelines on the same work is the experiment that would.

4.5.3 Where the gain comes from: a cumulative ablation

About five changes separate the October 2026 pipeline from the current one. To attribute the gain, the rerun adds them one at a time on the M2 Air and the A14, at 3 s and 30 s, under the same protocol (Table 7). Three host-code changes (parallel hn-nsf, vectorized sine, parallel iSTFT tail) have no runtime switch, so every rung, R0 included, runs them. The hypothesis, written into the draft of this paper before the rerun, is that admitting the generator to the ANE (R2→R3) accounts for most of the gain; if it accounts for well under half, the paper's claim is that folding admits the

vocoder while pipeline engineering supplies most of the speed. Steps that could not be isolated are merged with their neighbor and marked.

Table 7: Cumulative ablation, warm medians (ms). Each row adds one change to the row above; R4 replaces R3's generator. †On the current host code (parallel hn-nsf, vectorized sine, parallel iSTFT tail), which has no runtime switch; R0 is therefore faster than the October 2026 run itself. The M2 Air rows are upper bounds (load 3–5). §Ended above a nominal thermal state (R0 "serious", R1 and R2 "fair"): the old generator's 3.8 s calls heat the A14 within a cell.

	Change	M2 Air 3s	M2 Air 30s	A14 3s	A14 30s
R0	October 2026 packages (GPU generator)†	122.5	1162.8	410.1	3829.2§
R1	+ duration and F0Ntrain split	115.3	1118.6	395.2	3715.2§
R2	+ DecoderPre in ANE layout	115.5	1090.9	394.1	3678.9§
R3	+ folded generator (all ANE)	72.2	821.3	98.5	1555.7
R4	hybrid generator instead (ANE + GPU)	73.0	377.4	98.2	479.9

By that rule the A14 supports the hypothesis and the M2 Air does not. On the A14 at 30 s, admitting the generator to the ANE (R2→R3) supplies 63% of the total gain, the hybrid split (R3→R4) 32% and the front-stage changes (R0→R2) 4.5%; at 3 s, folding alone supplies 95%. On the M2 Air at 30 s, folding supplies 34% and the hybrid split 57%. On the M2 Air the alternative holds: folding admits the vocoder, and pipeline engineering — splitting the generator between the engines — supplies most of the speed. Folding is still necessary on both devices, because the hybrid's convolutions are folded ANE functions. Under the old pipeline the A14 also heats: its R0 30 s cell started at a nominal thermal state and ended "serious", while the hybrid's cell started and ended nominal.

Placement is checked, not assumed. On the M2 Air, Core ML's compute plan (`MLComputePlan`) assigns all 53 generator convolutions to the ANE in both the folded (R3) and hybrid (R4) packages. On the A14, on device, the folded 30 s package places 1,927 of its 1,928 ops on the ANE, and all 53 hybrid functions place their 83 ops on the ANE, none on the CPU or GPU.

4.5.4 Folded or hybrid?

Table 8: Folded vs hybrid generator, 30 s, per device. A tie is a warm-median difference within 5% or overlapping run-to-run ranges. Below 7 s the hybrid configuration runs the folded generator, so shorter buckets tie by construction and are omitted. Those identical 3 s cells differ by 0.3–1.7%; on the A14, two same-pipeline cells at the original ten-minute cooldown differed by 6.7%, so its noise floor is at least that. Every 30 s gap exceeds both. RAM is peak resident set size on Macs and peak `phys_footprint` on iPhones, in MB. GPU busy share is the GPU's occupancy of the generator's span, from one separate instrumented launch per phone, not from the timed runs. Mac rows are upper bounds (load gate missed).

Device	Folded	Hybrid	Δ	RAM fold	RAM hybrid	GPU busy (hybrid)	Verdict
M2 Ultra	706.4	158.9	−78%	415	434	–	hybrid
M2 Air	821.3	377.4	−54%	393	376	–	hybrid
M1 Mini	1091.7	308.5	−72%	404	375	–	hybrid
A17 Pro	780.9	318.5	−59%	780	1338	71%	hybrid
A14	1555.7	479.9	−69%	284	831	71%	hybrid

The result is not close: the hybrid is faster at every bucket of 7 s and longer on all five devices, by 16–78%, with no overlapping ranges, no crashes and higher SNR (45.4–45.7 against 43.7–44.0 dB on the 7–30 s inputs, measured on the M2 Ultra), so it ships as the default under the rule. Its costs are measured too. On the phones it raises peak memory (831 against 284 MB on the A14, 1,338 against 780 MB on the A17 Pro), while on the Macs the two are within 10% of each other. It keeps the GPU 61–72% busy during the generator, so an app that needs the GPU at the same moment — rendering, a game, another model — is better served by the folded generator, which uses no GPU and is the only form that runs when Metal is unavailable. The executor falls back to it on any hybrid failure.

4.5.5 Stage breakdown

Table 9: Per-stage times (ms), 30 s input, median of ten warm calls, same session. R0 is the October 2026 packages on the current host code (Table 7). hn-nsf runs on the CPU concurrently with DecoderPre, so stages do not sum to wall time. M2 Air rows are upper bounds (load 3–5).

Device	Pipeline	Duration	F0Ntrain	DecoderPre	hn-nsf	Generator	Wall
M2 Air	R0 (October packages)	64.0	32.7	32.7	17.4	1029.6	1162.8
M2 Air	shipped (hybrid)	54.2	25.1	23.6	22.0	265.6	377.4
A14	R0 (October packages)	154.2	51.1	59.4	61.9	3550.6	3829.2
A14	shipped (hybrid)	50.8	23.6	26.5	23.8	378.1	479.9

The generator fell from 92.7% of wall time to 78.8% on the A14 (3,551 to 378 ms) and from 88.5% to 70.4% on the M2 Air. The front stages also got faster — duration from 154 to 51 ms on the A14 with the split — and together they are now about 100 ms on both devices, 21–27% of wall time and the next lever. On a quiet M2 Air the hybrid's 30 s wall time was 223 ms (generator about 151 ms) in a lab run the day before (lab packages, not the frozen set), which bounds how much of the loaded row is host noise.

4.5.6 How close to the hardware?

We estimate a floor for the 30 s input from first principles: every convolution and matrix multiply on the ANE at its measured marginal rate (from per-function fits $t = a + b \cdot \text{GFLOP}$: M1 ≈9.8 TFLOPS, A14 4.2–5.2, A17 Pro 7–9), elementwise and AdaIN traffic hidden under it on the GPU, and the sequential LSTMs on the CPU (≈30 ms). Because the hybrid moves about 8 GB through DRAM per 30 s call, a second floor takes the maximum of the compute time and bytes ÷ bandwidth. We have not recomputed the floors against the frozen build; in lab runs on the same hybrid generator the pipeline sat at roughly 62–80% of them. The remaining gap is mostly a fixed cost per ANE call that is I/O rather than dispatch (fusing three convolutions into one call saved 0.5–0.85 ms per removed call; a k3 convolution plus residual moves about 117 MB in 6 ms on the A14), plus fold overhead (1.78 vs 1.35 TFLOP, from halos and row padding) and ANE/GPU hand-offs. We estimate the exact model cannot reach much above 75–85% of the floor with public APIs; exceeding 90% would require moving fewer bytes than the model does, for example a distilled generator, which would break the exactness contract. These are estimates, not measurements.

4.5.7 The private API buys almost nothing

Core ML's own calls into Espresso's private `e5rt` C API can be traced and replayed directly, from a command-line tool or a sandboxed iOS app, with output bit-identical to Core ML. In lab measurements it saves at most about 0.2 ms per ANE call (A14 k3 convolution: 5.91 ms Core ML, 5.70 ms e5rt), because the calls are bound by ANE I/O, which the private path does not change. In the rerun it lowered peak memory at 30 s by 15–23% on the M2 Ultra and the phones (A14: 643 against 831 MB) and by 3–4% on the M2 Air and M1, and it did not reliably lower latency: at 7 s and longer, e5rt was slower on the A17 Pro and 32–64% slower on the M2 Ultra, within 6% of the public path on the A14 and M1, and 39–60% faster at 10–30 s only on the M2 Air, the noisiest host. An app that uses it cannot ship in the App Store and may break with any OS update; we report it in one labeled column (Table 4) and use only the public path for every claim. The SDK compiles it out by default.

4.5.8 Control: no valid monolithic export

The null hypothesis against decomposition is one Core ML package for the whole model. With coremltools 8.3 it fails at two seams the pipeline already cuts. The full graph does not convert: duration-driven `repeat_interleave` has no static lowering (`ValueError: Cannot add const [None]`), and `torch.jit.trace` "succeeds" only by baking one utterance's alignment into constants — the traced graph crashes on a same-shape input with different durations. The strongest partial monolith, everything downstream of alignment, converts and produces wrong audio: correlation 0.27 in FP16 and 0.25 in FP32 against PyTorch, from Core ML's execution of SineGen's cumulative phase. For this model, decomposition was the precondition for any correct Core ML artifact we could build.

4.5.9 Audio quality

Every package is gated on SNR over the valid region against fp32 PyTorch; the frozen generators score 40.8–44.1 dB (folded) and 40.8–45.8 dB (hybrid) per clip against the fp32 PyTorch generator on the same inputs, above the 40 dB gate fixed before the run. The worst clip is the shortest input, "Hi." (1.2 s); numbers score 44.1/45.8 dB and heavy punctuation 43.4/44.9 dB (folded/hybrid). The split duration model predicts the same frame counts as the unsplit one on all eight clips. SNR alone has failed us before: in July 2026 a blind model-judged listening evaluation (Gemini, paired lineups with a known-bad control) caught three defects that every numerical gate had passed — a post-process that silenced real speech near punctuation, an inverse STFT at half amplitude for interior bins, and an RNG with an absorbing zero state. For the frozen build, the author listened to five clips (seed 42) covering the longest bucket, numbers, heavy punctuation, the shortest input and the folded generator, and heard no artifacts; per-clip hashes are archived (Appendix A). The plan called for the same automated judge; its cloud project failed on billing (HTTP 403), so the author listened instead. This is one unblinded listener, not a rated or blind study, and not a human MOS evaluation.

4.6 Limitations of Case Study 1

Regime. Phone numbers are sustained (three warm-ups, then ten back-to-back calls). A single request after idle is slower: the A14 generator pays a cold penalty after any break from its own work, which we could not remove with public or private APIs. Short bursts are faster than the sustained median. The rerun's "cold" call follows three preflight calls and lands within a few percent of the

warm median (A14 hybrid 30 s: 485 vs 480 ms); it does not measure the after-idle penalty, which we did not re-measure on the frozen build.

OS state. The flexible-length GPU generator of the October pipeline can abort inside MPS-Graph on macOS 27.0.1; it is disabled on OS 27 and not used here.

No concurrent-workload benchmark. We have not measured synthesis while a local LLM uses the GPU. In a lab run on an earlier build, a GPU blit kernel moving 51 GB/s on an M1 slowed the hybrid generator by 34%; the blit also takes GPU time, so this mixes DRAM and GPU contention, and the frozen build has not been measured under load.

Coverage. One vendor; M1, M2 and M2 Ultra Macs; A14 and A17 Pro phones.

5 Case Study 2: Magenta RealTime 2 Under a 40 ms Deadline

MRT2 is Kokoro's structural complement. Kokoro renders a finished utterance once; MRT2 generates one frame every 40 ms indefinitely, frame $N+1$ depends on frame N's tokens, and success is a sustained zero-underrun stream with p99 frame time under the deadline. It also has the motif Kokoro could route around: a stateful transformer (12 temporal layers, 41-frame local-attention K/V caches) plus a 12-level serial depth sampler on the hot path.

Pipeline and setup. Per frame: host conditioning → Core ML temporal step → Core ML depth rollout (all 12 RVQ levels in one prediction) → host codebook summation → Core ML codec decoder → host iSTFT and overlap-add → lock-free ring → `AVAudioSourceNode` (Figure 3). Audio-rate tensors never enter a Core ML graph: the codec stops at STFT-rate features — the geometry problem Kokoro's vocoder had to be folded to escape is designed out here. Measurements are on the same two iPhones, iOS 26.5, after numerical-parity gates against the MLX reference (transformer correlation > 0.9999, codec SNR ≥ 115 dB); placement claims come from `MLComputePlan` and Instruments, never from the requested policy.

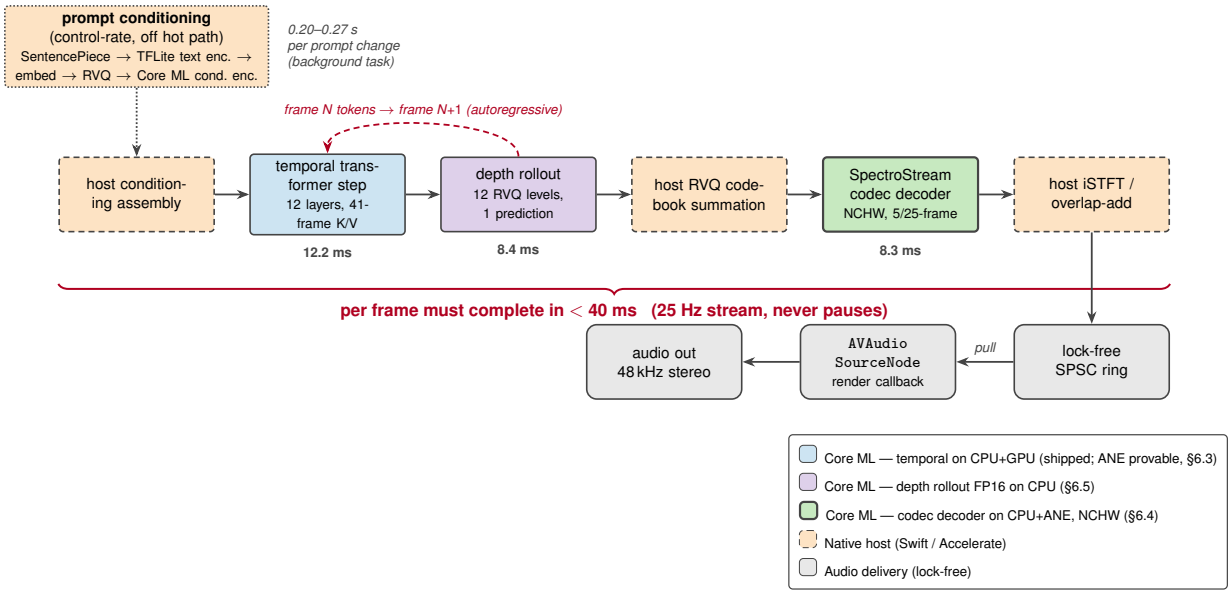


Figure 3: The MRT2 per-frame streaming loop. Each frame must finish within 40 ms. The three Core ML stages ship on three engines (temporal on GPU, depth rollout on CPU, codec decoder on ANE; per-stage p50 on the A17 Pro shown). The render callback never blocks on the model.

5.1 State Mutation, Not Attention, Is the ANE Cliff

Core ML's mutable state (`MLState`) holding the K/V caches fails twice. A one-frame stateful step compiles with an ANE island, but every multi-frame unroll fails ANE compilation (`ANECCompile() FAILED`, error −14) on both phones; and the one-frame graph that compiles is slower under streaming pressure than the same graph on the CPU (p99 29.7 vs 14.7 ms on the A14). A first host-owned-cache export that still assembled updates in-graph also failed. "Streaming transformers don't fit the ANE" was the available conclusion.

Instead we ran a falsification ladder on the A14: strip the graph to one suspect mechanism, prove it on device, add the next back (Table 10, Figure 4).

Table 10: Falsification ladder, iPhone 12 Pro (A14), FP16, p99 per call; every rung has ANE cost share 1.000.

Graph	ANE	CPU only	CPU+GPU
12-layer FFN stack, stateless	7.3 ms	8.9 ms	14.9 ms
12-layer attention math, no state	6.4 ms	6.5 ms	6.8 ms
1 layer + cache reads as inputs	1.9 ms	2.1 ms	11.9 ms
1 layer + current-token K/V update outputs	1.9 ms	2.2 ms	10.6 ms
4 layers, caches in / updates out	5.2 ms	6.5 ms	11.6 ms
8 layers, caches in / updates out	10.0 ms	12.1 ms	20.1 ms
12 layers, 48 caches in / 48 updates out	**15.0 ms**	15.4 ms	26.7 ms

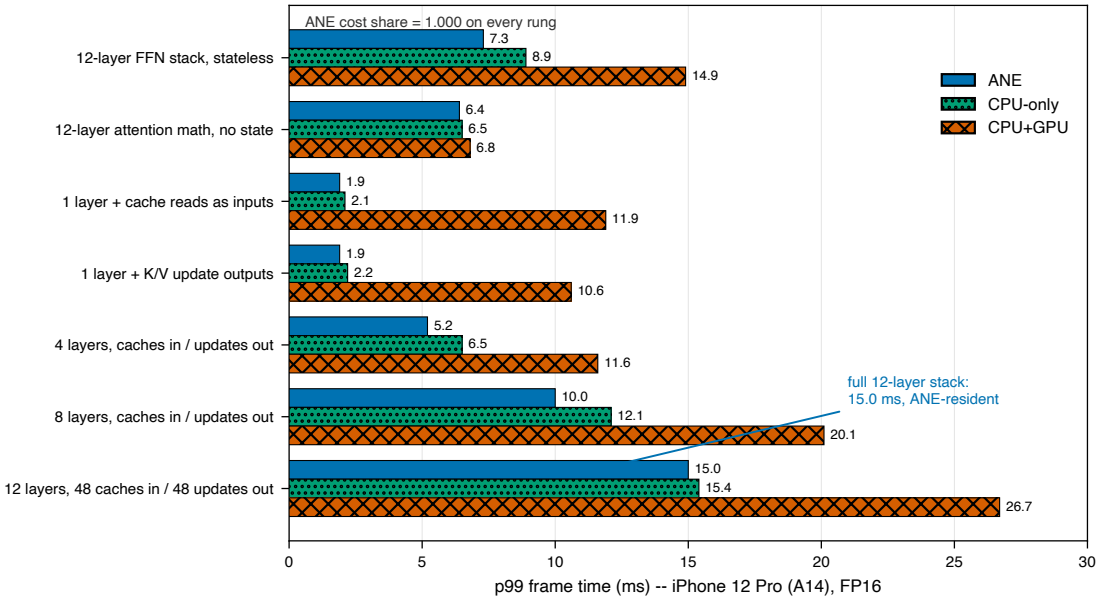


Figure 4: The falsification ladder (A14, FP16, p99). Every rung is ANE-clean and the ANE beats CPU-only and CPU+GPU, up to the complete 12-layer stack with all caches read in and all updates written out. The one thing that must be absent is in-graph cache mutation.

Every rung is ANE-resident, including the complete temporal stack with all 48 caches in and

48 one-token updates out. Attention, softmax, concatenation against caches and update outputs are all admissible; the state write is the cliff. For the taxonomy, the recurrence in a streaming transformer is cache bookkeeping — host logic — around per-step math that is Dense-Static. Cut there and the motif assignment inverts.

Admission is also fragile in a way the ladder does not show. The stateless artifact that ran a 10-minute zero-underrun stream in a headless host later fell back to the CPU (≈640 ms/frame) inside the full app, and the numerically correct rolling-cache graph that replaced it during a correctness overhaul (§5.4) mutates state in-graph and fails on the A14, consistent with the finding. The temporal stage therefore ships on `.cpuAndGPU` today; placement must be re-proven in the shipping process, not per artifact.

5.2 Layout Decides Numerical Survival

The SpectroStream decoder is textbook Dense-Static, and its naive FP16 export compiled and produced non-finite output. FP32 was clean but ran entirely on the CPU. Converting the upsampling block to channels-first internally, keeping the public I/O layout, fixed it — on one engine (Table 11).

Table 11: SpectroStream decoder placement and FP16 output health, iPhone 12 Pro.

Decoder artifact	Policy	p99	Output finite?
5-frame, channels-last	any	—	no (0.71 finite)
5-frame, NCHW	ANE (cost share 1.000)	**6.7 ms**	**yes**
5-frame, NCHW	CPU only	27.8 ms	no
5-frame, NCHW	CPU+GPU	32.2 ms	no
25-frame, NCHW	ANE (cost share 1.000)	**24.8 ms**	**yes**
25-frame, NCHW	CPU only	152.7 ms	no

Channels-first layout is documented vendor guidance for ANE throughput [6]. What we have not seen documented is the numerical coupling: **the ANE was the only engine that produced finite output from the same FP16 artifact.** Our hypothesis is that each engine lowers the same MIL program differently; we did not inspect the lowerings.

5.3 Weight Bandwidth Sets the Graph Boundary

The depth transformer samples 12 RVQ levels serially per frame. Twelve small predictions per frame cost about 40–45 ms on the A14, over the whole budget, whether each call computes twelve positions or one. Every prediction streams the stage's weights from DRAM: 97 MB ÷ 3.4–3.8 ms ≈ 26–29 GB/s, the A14's bandwidth; the 364 MB temporal graph gives the same rate. **Per-call cost ≈ weight bytes ÷ DRAM bandwidth**, the ordinary roofline bound, and since all engines share one DRAM no placement escapes it. Moving the 12-level rollout inside one prediction — with host-supplied Gumbel noise [24] and inverse temperature so sampling stays host-seeded and deterministic — streams the weights once per frame: 37 ms FP32 → **12.7 ms FP16 on the A14, 8.4 ms on the A17 Pro**. The FP32 export is token-exact (0/900 mismatches); FP16 flips near-tie tokens and shipped after listening gates. The same bound reappears in Kokoro: the hybrid generator's ANE calls are bound by I/O, not by dispatch (§4.5.6).

5.4 Host Glue Can Destroy Correctness

Three bugs shipped in early device builds while every model-level validation passed. Core ML pads output rows for 64-byte alignment, and a Swift sampler that indexed by `count / 12` read level k's logits $10k$ positions off — coarse structure sounded plausible, fine detail was scrambled. The runtime never wrote the temporal feedback input and made one depth prediction per frame instead of twelve. And a stateful graph's baked attention bias masked all 41 history slots, so it wrote state every frame and never read it; parity tests pass on write-only state. These are the source of the procedure's rules (step 6): validate every host consumer of a model output, read through strides, and prove a stateful graph reads its state.

5.5 Placement, Power and Status

In the earlier all-ANE configuration, Instruments traces on both phones show ANE intervals for all three model families and no app-attributed GPU intervals, encoded as a machine-checked placement artifact. Paired Power Profiler captures on the A14, varying only the temporal stage's policy, show process GPU impact 0.000 and 48.1 B CPU instructions over 60 s for ANE, against 2.23 and 110.3 B for the GPU control (Figure 5); the ANE arm slept 43% of the run under backpressure. These captures predate the correctness fixes: both arms ran identical semantics, so the comparison stands, but absolute numbers do not transfer.

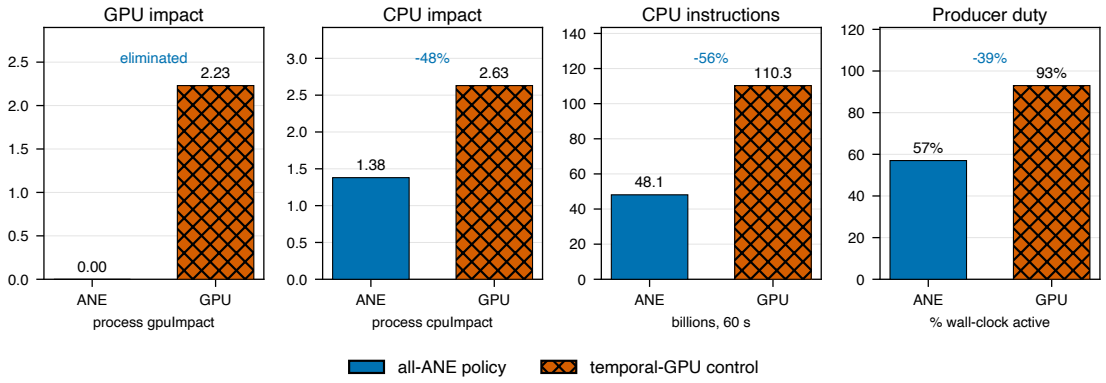


Figure 5: Paired power comparison on the A14, all-ANE temporal policy vs a temporal-GPU control, identical graphs, 60 s. The ANE policy removes process GPU impact (2.23 → 0.000) and cuts CPU instructions by 56%.

Status, plainly. With the corrected pipeline (temporal on GPU, depth rollout FP16 on CPU, decoder on ANE), the iPhone 15 Pro Max runs faster than real time: about 29 ms/frame p50, zero underruns over bounded 45 s runs, 1.37× headroom. The A14 does not hold the budget (about 50 ms/frame; temporal plus depth move about 0.7 GB per frame, 67% of its bandwidth). The 10-minute foreground soak fails on the A17 Pro: after banking about 20 s of lookahead, it throttles to about 0.95× real time around minutes 5–7. The named fixes — the temporal stack back on the ANE via the stateless boundary, and fewer temporal weight bytes — follow from the findings above and are not yet demonstrated.

6 Discussion

6.1 Structure Decides Admission

Across the two workloads, three structural properties governed ANE admission, and we found none of them documented as such by the vendor:

1. **Tensor geometry** (§4.2–4.3): audio-rate axes exceed per-axis limits. Folding the axis into rows, with exact global statistics, admitted the whole vocoder. MRT2 designed the problem out by stopping its codec at feature rate.

2. **In-graph state mutation** (§5.1): every graph that mutated K/V state inside Core ML failed compilation; the same math as a stateless step compiled and beat the CPU and GPU.

3. **Layout-coupled FP16 numerics** (§5.2): one FP16 program was finite only on the ANE, and only after a channels-first rewrite.

Op-level rewrites — `Linear` to 1×1 convolution, normalization lowerings — changed nothing in either workload; the MIL compiler makes those choices itself. The developer's levers are shapes, state boundaries and layout, and only a per-stage on-device ablation reveals which one is binding. Admission is also not the end of the question: the admitted all-ANE vocoder was 2.2–4.4× slower than the hybrid at 30 s on every device, and on the M2 Ultra slower than October's GPU pipeline, until the stage was split by op class across both engines (§4.4). "Dense-Static" is the candidate set for an engine, and op-class cost inside a stage can still favor splitting it.

6.2 What Did Not Work

Failures shaped the pipeline as much as successes. In Kokoro: a monolithic export (§4.5.8); op-level ANE rewrites (§4.2); sampled AdaIN statistics (1–4 dB); FP16 pre-scaling of AdaIN variance (16 dB, subnormal flush); fusing the three parallel convolution branches into one ANE call (0.5–0.85 ms saved per call, at the cost of ANE/GPU overlap); `.fastPrediction` specialization (no change); private `e5rt` dispatch (about 0.2 ms per call); QoS and work-interval settings, pre-warm loops and GPU-to-ANE event chaining (no gain); moving DecoderPre or the LSTMs to the GPU (slower, or changed duration rounding and therefore output); splitting a long first sentence for earlier audio (switching generators cost more than it saved); and halos below 8 frames. In MRT2: `MLState` caches, twelve depth predictions per frame, and FP32 or channels-last decoders.

6.3 When Surgical Inference Is Worth It

The method costs real engineering: per-bucket exports with validation, native DSP, per-stage ablations, a disciplined benchmark harness. It pays when the workload is compute-bound and has distinct stages (vocoders, codecs, image decoders, speech encoders), when it must run across a range of devices — the margin is largest on base chips and phones, where GPU frameworks fail outright (in October 2026, PyTorch MPS failed the 15 s and 30 s inputs on every Mac and kokoro-ios was killed for memory on the 6 GB iPhone 12 Pro) — and when the GPU is needed for something else. It pays least for large language models, whose cost is weight bandwidth (§5.3), and for small models where framework overhead is already negligible.

6.4 Limitations

Single vendor. The motifs and procedure do not assume Core ML, but the admission catalog is Apple-specific; other NPU toolchains (QNN, LiteRT delegates) need their own ablations.

Instrumentation asymmetry. MRT2 has Instruments residency and paired power captures; Kokoro has `MLComputePlan` placement evidence on a Mac and on-device compute-plan placement on the A14, but no Instruments residency traces and no power captures yet.

Quality evidence is narrow. Both case studies rest on numerical gates plus listening checks: automated blind model-judged lineups with known-bad controls during development, and, for Kokoro's frozen build, one unblinded listener. Neither is a human MOS study.

Two workloads. "Structure decides admission" holds in both workloads we studied. It is an empirical regularity across two models, not a law.

7 Related Work

Per-stage Core ML deployment. Apple's Stable Diffusion port [1] ships separate packages per model, and Apple's ANE transformer guidance [6] documents channels-first layout for throughput, which §5.2 extends to numerical survival. whisper.cpp [12] runs Whisper's encoder through Core ML beside a Metal decoder, and WhisperKit [28] ships Whisper as per-stage Core ML models with per-stage compute units. Surgical Inference systematizes this practice; it adds the reshaping step for rejected stages, the admission findings that make per-stage ablation necessary, intra-stage ANE/GPU co-execution, and negative results.

Kokoro ports. laishere/kokoro-coreml [21] splits Kokoro's vocoder into several Core ML packages with partial ANE placement (§4.2); soniqo/speech-swift [33] ships a Core ML Kokoro with a fixed-length output; mlalma/kokoro-ios [26] is the MLX Swift port we compare against. Piper [29] and MeloTTS [27] target CPU inference; we did not benchmark them.

Reverse-engineering the ANE. Orion [20] and community guides [32] document per-axis limits, per-dispatch costs and preferred lowerings. We corroborate the geometry limit, find that their op-level rewrites do not change Core ML runtime, and add folding as an exact escape from the limit, the state-mutation cliff and layout-dependent FP16 survival.

Heterogeneous scheduling. StarPU [7] and Legion [8] map tasks across CPU/GPU systems; ExecuTorch [30] and LiteRT delegates partition graphs across mobile backends at runtime. Closest to our hybrid generator, μLayer [19] splits single layers across a phone's CPU and GPU, CoDL [17] co-executes inference on CPU and GPU, and Band [16] coordinates multiple DNNs across mobile processors; for LLMs, llm.npu [34], PowerInfer-2 [35] and HeteroLLM [10] split inference across a phone's NPU, GPU and CPU. These systems mostly target Android SoCs with programmable access to each processor. We work through Core ML, whose ANE admission is opaque and must be probed, and split a stage by op class between an NPU and a GPU over shared memory.

Live music on device. MRT2's upstream path is MLX on M-series Macs [23, 25]. We know of no prior real-time phone deployment of this model class, nor prior documentation of the state-mutation cliff; Apple's `MLState` documentation [2] does not specify ANE behavior for graphs that mutate state.

8 Conclusion

The ANE rejected Kokoro's vocoder, and the rejection turned out to be about one axis's length. Folding that axis into rows, without changing the math, put the vocoder on the ANE; splitting

each stage by op class between the ANE and the GPU made it fast. The result synthesizes 27 s of speech in 309 ms on an M1 Mac Mini, 9.8× faster than MLX on the same machine, and in 319 ms on an iPhone 15 Pro Max, with about half of MLX's memory on the Mac and with the phone at the nominal thermal state where MLX, on the same work, reached throttling at 30 s. The same procedure carried a 230M-parameter streaming music model onto a phone, faster than real time in bounded runs though not yet in a 10-minute soak, and found the same pattern twice more: state mutation, not attention, was the compile cliff, and layout decided whether FP16 output was finite.

The useful question for on-device AI is not "which accelerator should run this model?" It is "what is the smallest set of static packages and native kernels — each reshaped until its engine admits it — that lets the whole chip do the work?"

Acknowledgments

Several improvements to the Kokoro pipeline came from outside contributors. Willem Kokke contributed mask-aware bucketing, which replaced the duration model's unrolled BiLSTM with stock LSTM operations and stopped bucket padding from biasing normalization statistics; he also removed the punctuation-span fade, corrected the generator's harmonic-input geometry and the STFT phase sign at the DC and Nyquist bins, ran the harmonic source concurrently, and contributed the flexible-length generator. Ahmed El Hadidi fixed a Swift error-mapping build failure, and David Bitton fixed the Intel (x86_64) SDK build. We thank them for their work and their careful measurements.

We also thank Apple's Core ML team. In August 2025, Kokoro's LSTM and F0/N stages crashed intermittently inside Core ML's CPU compute kernels (`libBNNS`, called from Espresso's E5RT runtime), even with inputs that matched the model's contract. No change we could make to the export avoided it; removing those stages from the graph reduced the crashes but did not end them, and the work stalled until the fix. We reported the kernel bug, and Apple fixed it in macOS 26.2. The pipeline described here runs those LSTM stages on the CPU through Core ML.

References

[1] Apple. Stable diffusion with core ml on apple silicon. https://github.com/apple/ml-stable-diffusion, 2022.

[2] Apple. MLState — core ml stateful models documentation. https://developer.apple.com/documentation/coreml/mlstate, 2024.

[3] Apple. Core ml tools. https://github.com/apple/coremltools, 2025. Software package for converting trained models to Core ML; version 8.3.0 (April 2025) used for every export.

[4] Apple. Pytorch conversion workflow. https://apple.github.io/coremltools/docs-guides/source/convert-pytorch-workflow.html, 2026. Core ML Tools guide. Accessed October 2026.

[5] Apple Developer. Accelerated pytorch training on mac. https://developer.apple.com/metal/pytorch/, 2026. PyTorch MPS backend documentation. Accessed October 2026.

[6] Apple Machine Learning Research. Deploying transformers on the apple neural engine. https://machinelearning.apple.com/research/neural-engine-transformers, June 2022. Reference implementation: https://github.com/apple/ml-ane-transformers.

[7] Cédric Augonnet, Samuel Thibault, Raymond Namyst, and Pierre-André Wacrenier. StarPU: A unified platform for task scheduling on heterogeneous multicore architectures. *Concurrency and Computation: Practice and Experience*, 23(2):187–198, 2011.

[8] Michael Bauer, Sean Treichler, Elliott Slaughter, and Alex Aiken. Legion: Expressing locality and independence with logical regions. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC)*, 2012.

[9] Blaizzy. mlx-audio. https://github.com/Blaizzy/mlx-audio, 2026. Version 0.4.3, commit 862dfbe (June 2026); MLX audio generation toolkit used for the MLX Kokoro baseline.

[10] Le Chen, Dahu Feng, Erhu Feng, Yingrui Wang, Rong Zhao, Yubin Xia, Pinjie Xu, and Haibo Chen. Characterizing mobile SoC for accelerating heterogeneous LLM inference. https://arxiv.org/abs/2501.14794, 2025. Version 1 titled "HeteroLLM: Accelerating Large Language Model Inference on Mobile SoCs platform with Heterogeneous AI Accelerators"; the engine is called HeteroInfer in later versions.

[11] Georgi Gerganov and contributors. llama.cpp. https://github.com/ggml-org/llama.cpp, 2026. Software. Accessed October 2026.

[12] Georgi Gerganov and contributors. whisper.cpp. https://github.com/ggml-org/whisper.cpp, 2026. Software; includes a Core ML encoder path. Accessed October 2026.

[13] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on apple silicon. https://github.com/ml-explore/mlx, 2023. Software.

[14] hexgrad. Kokoro-82m. https://huggingface.co/hexgrad/Kokoro-82M, 2024. Open-weight 82M-parameter text-to-speech model; model repository created December 2024.

[15] hexgrad. kokoro. https://github.com/hexgrad/kokoro, 2025. Python inference library for Kokoro-82M; repository created January 2025.

[16] Joo Seong Jeong, Jingyu Lee, Donghyun Kim, Changmin Jeon, Changjin Jeong, Youngki Lee, and Byung-Gon Chun. Band: Coordinated multi-DNN inference on heterogeneous mobile processors. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services (MobiSys)*, pages 235–247, 2022. doi: 10.1145/3498361.3538948.

[17] Fucheng Jia, Deyu Zhang, Ting Cao, Shiqi Jiang, Yunxin Liu, Ju Ren, and Yaoxue Zhang. CoDL: Efficient CPU-GPU co-execution for deep learning inference on mobile devices. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services (MobiSys)*, pages 209–221, 2022. doi: 10.1145/3498361.3538932.

[18] Takuhiro Kaneko, Kou Tanaka, Hirokazu Kameoka, and Shogo Seki. iSTFTNet: Fast and lightweight mel-spectrogram vocoder incorporating inverse short-time fourier transform. In *ICASSP*, 2022. https://arxiv.org/abs/2203.02395.

[19] Youngsok Kim, Joonsung Kim, Dongju Chae, Daehyun Kim, and Jangwoo Kim. μLayer: Low latency on-device inference using cooperative single-layer acceleration and processor-friendly quantization. In *Proceedings of the Fourteenth EuroSys Conference (EuroSys)*, pages 1–15, 2019. doi: 10.1145/3302424.3303950.

[20] Ramchand Kumaresan. Orion: Characterizing and programming apple's neural engine for LLM training and inference. https://arxiv.org/abs/2603.06728, 2026.

[21] laishere. kokoro-coreml. https://github.com/laishere/kokoro-coreml, 2026. Commit 484907d; independent Core ML Kokoro implementation used as a comparison baseline. Accessed October 2026.

[22] Yinghao Aaron Li, Cong Han, Vinay S. Raghavan, Gavin Mischler, and Nima Mesgarani. StyleTTS 2: Towards human-level text-to-speech through style diffusion and adversarial training with large speech language models. In *NeurIPS*, 2023. https://arxiv.org/abs/2306.07691.

[23] Lyria Team, Antoine Caillon, Brian McWilliams, Cassie Tarakajian, Ian Simon, Ilaria Manco, Jesse Engel, Noah Constant, et al. Live music models. https://arxiv.org/abs/2508.04651, 2025. Introduces Magenta RealTime.

[24] Chris J. Maddison, Daniel Tarlow, and Tom Minka. A* sampling. In *NeurIPS*, 2014. https://arxiv.org/abs/1411.0030.

[25] Magenta team, Google. magenta-realtime. https://github.com/magenta/magenta-realtime, 2026. Open-weights live music model repository; upstream source for the second case study. Accessed October 2026.

[26] mlalma. kokoro-ios. https://github.com/mlalma/kokoro-ios, 2025. Version 1.0.8 (October 2025); MLX Swift Kokoro implementation used for iOS comparison.

[27] MyShell.ai. MeloTTS. https://github.com/myshell-ai/MeloTTS, 2024. Open-source multilingual text-to-speech system; repository created February 2024.

[28] Atila Orhon et al. WhisperKit: On-device real-time ASR with billion-scale transformers. https://arxiv.org/abs/2507.10860, 2025. Software: https://github.com/argmaxinc/WhisperKit.

[29] Piper contributors. Piper. https://github.com/rhasspy/piper, 2023. Fast local neural text-to-speech system; repository created January 2023, since archived.

[30] PyTorch. ExecuTorch. https://github.com/pytorch/executorch, 2026. On-device inference runtime with delegate backends. Accessed October 2026.

[31] PyTorch. MPS backend notes. https://pytorch.org/docs/stable/notes/mps.html, 2026. Apple Silicon GPU backend documentation. Accessed October 2026.

[32] skyfallsin. Apple neural engine reverse-engineering field guide. https://github.com/skyfallsin/apple-neural-engine-field-guide, 2026. Experiment-driven ANE notes and constraints. Accessed October 2026.

[33] soniqo. speech-swift. https://github.com/soniqo/speech-swift, 2026. Commit 0d09a2e; independent Core ML Kokoro implementation used as a comparison baseline. Accessed October 2026.

[34] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. Fast on-device LLM inference with NPUs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 445–462, 2025. doi: 10.1145/3669940.3707239.

[35] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. PowerInfer-2: Fast large language model inference on a smartphone. https://arxiv.org/abs/2406.06282, 2024.

[36] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. SoundStream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30:495–507, 2022. doi: 10.1109/TASLP.2021.3129994. https://arxiv.org/abs/2107.03312.

A Artifact and Reproduction Map

A.1 Case Study 1 (Kokoro-82M)

- Swift runtime pipeline and staged policy: `swift/Sources/KokoroPipeline/` (`KokoroPipeline.swift`, `KokoroSynthesisExecutor.swift`)
- Folded generator host code (fold, masks, unfold, iSTFT tail): `swift/Sources/KokoroPipeline/FoldedGenerator.swift`; exporter and parity: `scripts/probe_generator_ane_fold.py`
- Hybrid generator (ANE functions, Metal kernels, shared tensors): `swift/Sources/KokoroPipeline/HybridGenerator.swift`, `HybridKernels.swift`, `HybridArenaLayout.swift`; exporter: `scripts/export_hybrid_generator.py`
- Frozen build: source commit `903f1fc1`; its 68 Core ML packages, byte-identical, at Hugging Face revision `e65729b9` under `coreml/` and `coreml-{dur-ane,f0n-split,dpre-ane,fold,hybrid}/`
- Frozen rerun (plan 014): `README/Plans/014-paper-cs1-hybrid-frozen-rerun-plan.md`; package manifest with SHA-256 for 68 packages and both iPhone apps: `paper/data/frozen_rerun_014/manifest.json`; quality and per-call timing records: `paper/data/frozen_rerun_014/`
- Split duration: `swift/Sources/KokoroPipeline/SplitDuration.swift`; harmonic source: `HarmonicSource.swift`
- Private e5rt path (aside only): `swift/Sources/KokoroPipeline/E5Runtime.swift`
- ANE lab (per-primitive timing app and driver): `tools/ane-lab/`
- Generator investigation log, op admission, numerics, rejected experiments: `README/Notes/ane-folded-generator.md`
- Frozen rerun plan, protocol and results: `README/Plans/014-paper-cs1-hybrid-frozen-rerun-plan.md`, `README/Notes/paper-cs1-hybrid-rerun-2026-10-23.md`
- Mac harness: `swift/Sources/KokoroBenchmark/`, `scripts/bakeoff_harness.py`; iPhone app: `ios-bench/`
- October 2026 pipeline run (Appendix C): `README/Notes/paper-cs1-frozen-rerun-2026-10-20.md`
- Monolithic-export control: `README/Notes/monolithic-coreml-control-experiment.md`; July 2026 listening evaluation: `README/Notes/cs1-audio-quality-evaluation-2026-07-14.md`

A.2 Case Study 2 (Magenta RealTime 2)

The public magenta-realtime-2-iphone repository holds the corrected exporters (stateless temporal boundary, in-graph FP16 depth rollout, finite-FP16 channels-first decoder), the superseded exporters as negative-result artifacts (they reproduce the `ANECCompile` −14 rejection and the non-finite decoder), parity validators with receipts (including the 0/900 token-exactness result), fixtures and `MODELS.md`, which states per artifact what is uploaded versus reproducible from an exporter. The streaming runtime, on-device probe, residency-proof runner, paired power capture and sustained-run analyzer live in the private Crossfade fork of `magenta/magenta-realtime` (available from the author).

B Benchmark Boundaries

Kokoro: phoneme token IDs and voice embedding in, 24 kHz PCM resident in memory out. Model download, process startup, first-use compilation, playback and text-to-phoneme conversion are excluded unless a comparison's public API forces a wider boundary; kokoro-ios takes raw text, so its times include a G2P pass (0.17–1.05 ms per input in isolation). MRT2: per-stage times at the Core ML prediction call (warmed, p50/p99, with placement evidence from the same run); end-to-end claims at the audio contract (frames into and out of the render ring, underruns, ring watermarks and thermal state logged per iteration).

C The October 2026 Pipeline

The previous pipeline ran the generator on the GPU in fixed buckets and is the ablation's R0. Its October 2026 run (commit `d32ef0d3`, Hugging Face revision `dadebca4`; M2 Ultra on macOS 26.7.1, others on 27.0.1; the M2 Air never cleared the load gate) compared it with every baseline we could build. We keep these results because the rerun does not repeat the PyTorch arms, and we do not mix sessions in any ratio.

Table 12: October 2026 pipeline (F_{Oct}) vs baselines, warm medians (ms), same session per machine. A: Python prefix + Core ML vocoder; D: PyTorch MPS; E: PyTorch CPU. "err": MLX broadcast-shape error (3 s) or MPS 65,536-channel limit (15 s, 30 s).

Machine	Input	F_{Oct}	MLX	A	D	E
M2 Ultra	3s	44.5	err	124	193	259
M2 Ultra	30s	286.5	685.3	631	err	1921
M2 Air	3s	123.4	err	236	367	615
M2 Air	30s	1175.5	2150.0	3381	err	6114
M1 Mini	3s	205.3	err	281	553	654
M1 Mini	30s	1635.5	3003.7	2066	err	6487

Across all five buckets the October pipeline was 1.8–2.4× faster than MLX, 2.7–6.5× faster than PyTorch MPS where MPS completed, and 3.2–6.8× faster than PyTorch CPU. On the iPhones (iOS 27.0.1, flexible GPU generator) it took 1,721 ms (A17 Pro) and 4,017 ms (A14) for the 30 s input, 2.6–3.2× and 2.0–2.2× faster than kokoro-ios; kokoro-ios was killed for memory at 30 s on the A14. laishere/kokoro-coreml's chain-only boundary excludes G2P and feed preparation; on the M2 Ultra the October pipeline was 1.9–2.0× faster, and its converter aborts on macOS 27.0.1. Full tables: `README/Notes/paper-cs1-frozen-rerun-2026-10-20.md`.